DAA Lab 1

Experimental Analysis – Sorting and Fibonacci Algorithms

Purpose. This document presents the measured execution time and peak memory usage for the implemented sorting and Fibonacci algorithms. The experiments use the input sizes and conditions specified in the assignment.

Experimental setup. The notebook uses Python's *time.perf_counter()* for elapsed time and *tracemalloc* for peak memory. Sorting uses 100, 500, 1,000, 5,000 and an attempted 10,000 elements, with sorted, reverse-sorted and random inputs. Fibonacci uses n = 10, 20, 30 and 40.

Important limitation. The supplied sorting CSV contains complete results through 5,000 elements and only the first three 10,000 sorted cases (Bubble, Insertion and Merge). The 10,000-element Quick Sort and remaining 10,000 conditions were not completed. The notebook records that the 10,000-element experiment was interrupted because of excessive execution time. No missing values are invented in this report.

1. Sorting Results – Execution Time

n	Condition	Algorithm	Time (s)
100	Sorted	Bubble Sort	0.0199953
100	Sorted	Insertion Sort	4.6632e-05
100	Sorted	Merge Sort	0.00101785
100	Sorted	Quick Sort	0.000634399
100	Reverse Sorted	Bubble Sort	0.000591672
100	Reverse Sorted	Insertion Sort	0.000264394
100	Reverse Sorted	Merge Sort	0.000646288
100	Reverse Sorted	Quick Sort	0.000508814
100	Random	Bubble Sort	0.000487447
100	Random	Insertion Sort	0.000144189
100	Random	Merge Sort	0.000678056
100	Random	Quick Sort	0.00049925
500	Sorted	Bubble Sort	0.000781745
500	Sorted	Insertion Sort	0.000786463
500	Sorted	Merge Sort	0.00420599
500	Sorted	Quick Sort	0.107775
500	Reverse Sorted	Bubble Sort	0.119921
500	Reverse Sorted	Insertion Sort	0.0596808
500	Reverse Sorted	Merge Sort	0.00335612
500	Reverse Sorted	Quick Sort	0.173949
500	Random	Bubble Sort	0.113676
500	Random	Insertion Sort	0.0470139
500	Random	Merge Sort	0.00314703
500	Random	Quick Sort	0.00400569
1000	Sorted	Bubble Sort	0.00114918
1000	Sorted	Insertion Sort	0.00179584
1000	Sorted	Merge Sort	0.00815177
1000	Sorted	Quick Sort	1.50894
1000	Reverse Sorted	Bubble Sort	1.00131
1000	Reverse Sorted	Insertion Sort	0.518439
1000	Reverse Sorted	Merge Sort	0.00780963
1000	Reverse Sorted	Quick Sort	2.31488
1000	Random	Bubble Sort	0.860628
1000	Random	Insertion Sort	0.380641
1000	Random	Merge Sort	0.0100654

n	Condition	Algorithm	Time (s)
1000	Random	Quick Sort	0.0133369
5000	Sorted	Bubble Sort	0.00772161
5000	Sorted	Insertion Sort	0.0119396
5000	Sorted	Merge Sort	0.0588723
5000	Sorted	Quick Sort	343.899
5000	Reverse Sorted	Bubble Sort	38.8347
5000	Reverse Sorted	Insertion Sort	21.4492
5000	Reverse Sorted	Merge Sort	0.0582276
5000	Reverse Sorted	Quick Sort	282.145
5000	Random	Bubble Sort	37.9792
5000	Random	Insertion Sort	10.9122
5000	Random	Merge Sort	0.0828626
5000	Random	Quick Sort	0.0990471
10000	Sorted	Bubble Sort	0.014797
10000	Sorted	Insertion Sort	0.0253118
10000	Sorted	Merge Sort	0.146144

The 5,000-element results show the expected scalability differences: Merge Sort remains below 0.1 seconds for all three conditions, while Bubble and Insertion Sort become much slower on reverse/random inputs. Quick Sort is fast on random data (0.0990 s) but extremely slow on sorted (343.8992 s) and reverse-sorted (282.1451 s) inputs when the last element is used as pivot.

2. Sorting Results – Peak Memory

n	Condition	Algorithm	Memory (KB)
100	Sorted	Bubble Sort	1.099e+04
100	Sorted	Insertion Sort	0.8672
100	Sorted	Merge Sort	4.273
100	Sorted	Quick Sort	0.8672
100	Reverse Sorted	Bubble Sort	0.9062
100	Reverse Sorted	Insertion Sort	0.8672
100	Reverse Sorted	Merge Sort	3.289
100	Reverse Sorted	Quick Sort	0.8672
100	Random	Bubble Sort	0.9062
100	Random	Insertion Sort	0.8672
100	Random	Merge Sort	3.32
100	Random	Quick Sort	1.503
500	Sorted	Bubble Sort	4.121
500	Sorted	Insertion Sort	4.051
500	Sorted	Merge Sort	16.17
500	Sorted	Quick Sort	20.04
500	Reverse Sorted	Bubble Sort	4.152
500	Reverse Sorted	Insertion Sort	8.861
500	Reverse Sorted	Merge Sort	15.84
500	Reverse Sorted	Quick Sort	19.58
500	Random	Bubble Sort	7.082
500	Random	Insertion Sort	4.741
500	Random	Merge Sort	15.87
500	Random	Quick Sort	5.117
1000	Sorted	Bubble Sort	8.027
1000	Sorted	Insertion Sort	7.957
1000	Sorted	Merge Sort	31.5
1000	Sorted	Quick Sort	55.12
1000	Reverse Sorted	Bubble Sort	11.22
1000	Reverse Sorted	Insertion Sort	11.14

n	Condition	Algorithm	Memory (KB)
1000	Reverse Sorted	Merge Sort	31.5
1000	Reverse Sorted	Quick Sort	54.46
1000	Random	Bubble Sort	11.21
1000	Random	Insertion Sort	11.14
1000	Random	Merge Sort	32.22
1000	Random	Quick Sort	8.961
5000	Sorted	Bubble Sort	39.28
5000	Sorted	Insertion Sort	39.21
5000	Sorted	Merge Sort	157.1
5000	Sorted	Quick Sort	339.1
5000	Reverse Sorted	Bubble Sort	42.48
5000	Reverse Sorted	Insertion Sort	42.39
5000	Reverse Sorted	Merge Sort	157.1
5000	Reverse Sorted	Quick Sort	330.1
5000	Random	Bubble Sort	42.56
5000	Random	Insertion Sort	42.48
5000	Random	Merge Sort	159.7
5000	Random	Quick Sort	41.09
10000	Sorted	Bubble Sort	78.34
10000	Sorted	Insertion Sort	81.43
10000	Sorted	Merge Sort	313.5

Peak-memory measurements include Python/tracemalloc measurement effects and should be interpreted as experimental peak allocations rather than pure algorithm-only memory. Merge Sort generally shows higher auxiliary memory than the in-place sorting implementations, consistent with its O(n) extra-space requirement.

3. Fibonacci Results – Execution Time and Memory

n	Algorithm	Value	Time (s)	Memory (KB)
10	Recursive	55	1.5545e-05	8652.07
10	Iterative	55	9.63e-06	0.0859375
10	Dynamic Programming	55	8.33e-06	0.4375
20	Recursive	6765	0.0011444	0.15625
20	Iterative	6765	9.021e-06	0.132812
20	Dynamic Programming	6765	1.8345e-05	0.835938
30	Recursive	832040	0.167761	0.822266
30	Iterative	832040	2.052e-05	0.132812
30	Dynamic Programming	832040	4.2155e-05	1.94531
40	Recursive	102334155	20.2445	8.11426
40	Iterative	102334155	2.6434e-05	0.132812
40	Dynamic Programming	102334155	5.7607e-05	1.94531

Fibonacci observations. Recursive Fibonacci grows dramatically with n: 0.0000155 s at n=10, 0.001144 s at n=20, 0.167761 s at n=30, and 20.2445 s at n=40. Iterative and Dynamic Programming Fibonacci remain in the microsecond range for these tested values. DP uses more memory than the iterative two-variable method because it stores computed values and uses a recursion stack.

4. Key Experimental Findings

• Merge Sort is the most consistent sorting algorithm in the supplied measurements as input size increases.
• Bubble Sort and Insertion Sort show strong growth on reverse/random inputs at 5,000 elements.
• Quick Sort with a last-element pivot is highly sensitive to input order; the 5,000 sorted case took 343.8992 s and the reverse-sorted case took 282.1451 s, while the random case took only 0.0990 s.
• Recursive Fibonacci demonstrates the practical effect of exponential growth, reaching 20.2445 s at n=40.
• Iterative Fibonacci provides the lowest measured time and very low memory usage; DP is also fast but consumes additional memory.